1. Objective

This assessment evaluates your ability to leverage AI-powered agentic tools (GitHub Copilot, Claude, Cursor, or any equivalent) to perform end-to-end quality engineering tasks — from requirements analysis through automated test generation and execution. The goal is not just to produce test artefacts, but to demonstrate a modern, agent-assisted QA workflow where AI does the heavy lifting under your architectural direction.

---

2. Application Under Test

InvenTree — an open-source inventory management system built with Python/Django.

You will focus exclusively on the Parts module, which is the core domain entity in InvenTree. The Parts module covers part creation, categorisation, parameters, templates/variants, revisions, stock tracking, BOM management, and related API endpoints.

---

3. Problem Statement

Phase 1 — Requirements Analysis & Test Case Generation (UI) Using an

AI agent of your choice, perform the following:

1. Ingest the requirements from the InvenTree Parts documentation: https://docs.inventree.org/en/stable/part/

   (Include sub-pages: Part Views, Part Parameters, Part Templates, Part Revisions, Creating a Part, etc.)
2. Generate a comprehensive set of UI/manual test cases covering (at minimum):
   - Part creation (manual entry and import flows)
   - Part detail view — all tabs (Stock, BOM, Allocated, Build Orders, Parameters, Variants, Revisions, Attachments, Related Parts, Test Templates)
   - Part categories — hierarchy, filtering, parametric tables
   - Part attributes — Virtual, Template, Assembly, Component, Trackable, Purchaseable, Salable, Active/Inactive
   - Units of measure configuration
   - Part revisions — creation, constraints (circular references, unique codes, template restrictions)
   - Negative and boundary scenarios (e.g., duplicate IPN, inactive part restrictions, revision-of-revision prevention)

Phase 2 — API Specification Analysis & API Test Generation

1. Ingest the API schema from: https://docs.inventree.org/en/stable/api/schema/part/

2. Generate API test cases (manual) covering: CRUD operations on Parts and Part Categories
   - Filtering, pagination, and search on the Parts list endpoint
   - Field-level validation (required fields, max lengths, nullable constraints, read-only fields)
   - Relational integrity (category assignment, default locations, supplier linkage) Edge
   - cases (invalid payloads, unauthorised access, conflict scenarios)
   - 3. Generate API automation scripts using a framework of your choice (e.g., pytest + requests, Playwright API testing, REST Assured, Postman/Newman, Karate, etc.). Scripts should:
   - Be executable against a running InvenTree instance (Docker setup is acceptable)
   - Cover positive, negative, and boundary scenarios
   - Include assertions on status codes, response schema, and business logic
   - Demonstrate data-driven or parameterised testing where appropriate

Phase 3 — UI Automation Scripts

1. Using the test cases from Phase 1, generate UI automation scripts using a framework of your choice (e.g., Playwright, Cypress, Selenium, etc.). Scripts should:
   - Cover the core Part CRUD workflows
   - Validate key UI elements, navigation, and form behaviour
   - Include at least one cross-functional flow (e.g., create a part → add parameters → create stock → verify in category view)
   - Handle waits, selectors, and assertions robustly

---

4. Data Sources for the Agent

Source	URL
Parts Requirements	https://docs.inventree.org/en/stable/part/ (and sub-pages)
API Schema	https://docs.inventree.org/en/stable/api/schema/part/
Source	URL
InvenTree Demo (optional)	https://demo.inventree.org
InvenTree GitHub	https://github.com/inventree/InvenTree

You may feed these sources into your agent via URL ingestion, copy-paste, file upload, or any method supported by your tooling.

---

5. Expected Deliverables

A. Test Artefacts

Deliverable	Format
UI manual test cases	Spreadsheet, Markdown, or test management export
API manual test cases	Spreadsheet, Markdown, or test management export
UI automation scripts	Runnable project in chosen framework
API automation scripts	Runnable project in chosen framework

B. Agent Artefacts

Deliverable	Description
Agent prompts &	All prompts, system instructions, and markdown files used to guide the AI agent instructions
Agent configuration	Any `.cursorrules`, `.github/copilot-instructions.md`, `claude_project` instructions, or equivalent config files
Conversation logs	Exported chat logs or session history showing the agent interaction (optional)

C. Video Recording

A screen recording of the end-to-end flow demonstrating:

- The agent generating test cases from requirements
- The agent generating automation scripts from the API spec
- Test execution (at least a subset of tests running successfully)
- Any iterative refinement or correction you performed on the agent's output

---

6. Rules & Guidelines

- You may use any AI-agentic tool — GitHub Copilot, Claude (chat or Claude Code), Cursor, Windsurf, Aider, or any other. Using multiple tools in combination is encouraged.
- You may use any automation framework — the choice of language and tooling is part of the assessment.
- All generated code must be reviewable and runnable. If the agent produced code that needed fixes, document what you changed and why.
- Do not submit purely hand-written tests and claim they were agent-generated. The agent artefacts (prompts, logs) should demonstrate the agentic workflow.

- You may use the InvenTree Docker image for local testing: `docker compose up` from the InvenTree repository.

---

7. Submission Instructions

Submit a single ZIP or Git repository containing:

```
submission/
├── README.md              # Setup instructions, tool choices, approach summary
├── agents/                # Prompts, instructions, config files
│   ├── prompts.md
│   ├── system-instructions.md
│   └── <tool-specific configs>
├── test-cases/
│   ├── ui-manual-tests.md     # or .xlsx / .csv
│   └── api-manual-tests.md    # or .xlsx / .csv
├── automation/
│   ├── ui/                # UI automation project
│   │   ├── package.json / requirements.txt
│   │   ├── tests/
│   │   └── ...
│   └── api/               # API automation project
│       ├── package.json / requirements.txt
│       ├── tests/
│       └── ...
└── video/
    └── recording.mp4      # or link to cloud-hosted video
```